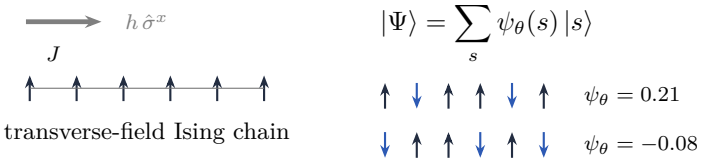
Project 3. A Neural-Network Quantum State for a Spin Chain

Use a neural network as the wavefunction of a genuine quantum many-body system and find its ground state by the variational principle. You will study the 1D transverse-field Ising chain of about 6 to 12 spins, optimise a neural quantum state with variational Monte Carlo, and benchmark the energy against exact diagonalisation. This reaches the many-body problem in its smallest honest form, the same method used in current research, on a system small enough to verify completely.



To learn the machine learning tools you will need PyTorch and gradient-based optimisation. Useful starting points are the PyTorch 60-minute blitz at https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html, the PyTorch autograd tutorial at https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html, the NetKet toolbox built for neural quantum states at https://www.netket.org/tutorials/, and the machine-learning-for-physicists notebooks of Mehta et al. (2019), https://doi.org/10.1016/j.physrep.2019.03.001.

On the physics side, any quantum or many-body text covers the transverse-field Ising model and the variational principle. The field is reviewed by Carleo et al., Machine learning and the physical sciences, Rev. Mod. Phys. 91 (2019), https://doi.org/10.1103/RevModPhys.91.045002, and founded by Carleo and Troyer, Solving the quantum many-body problem with artificial neural networks, Science 355 (2017), https://doi.org/10.1126/science.aag2302. A small-system worked example is given by Saito, Solving the Bose-Hubbard model with machine learning, https://arxiv.org/abs/1707.09723, and the NetKet 3 software is described by Vicentini et al. (2022), https://doi.org/10.21468/SciPostPhysCodeb.7.

If the basic project works, you can extend it toward real research in two directions. You can scale to system sizes beyond exact diagonalisation, or move to a 2D lattice or frustrated model where the method's value over traditional approaches becomes real. You can also swap the ansatz, going from a restricted Boltzmann machine to a convolutional or autoregressive network, and study how expressivity affects accuracy, which is an active research question.

For the timeline, spend the first two weeks learning PyTorch and NetKet, the next two weeks on exact diagonalisation of the chain for a reference energy, and the final two weeks optimising the neural quantum state and matching it. The outputs are the variational Monte Carlo code, energy-versus-iteration plots, and a table of machine-learning versus exact E_0 across system size N. Communicate via Microsoft Teams, posting questions, convergence plots and results there, and benchmark against exact diagonalisation while keeping a dated log of every run.

For the physics background, a quantum state of N spins lives in a Hilbert space of dimension 2^N,

$$|\Psi\rangle = \sum_s \psi(s)\,|s\rangle, \qquad s = (s_1, \ldots, s_N), \quad s_i = \pm 1, \tag{1}$$

so it is specified by one complex amplitude $\psi(s)$ for every spin pattern s. That list has 2^N entries and grows exponentially, which is why exact storage fails beyond a few tens of spins and why a

compact representation is needed. The model system is the transverse-field Ising chain,

$$\hat{H} = -J \sum_i \hat{\sigma}_i^z \hat{\sigma}_{i+1}^z \; - \; h \sum_i \hat{\sigma}_i^x, \tag{2}$$

where the $\hat{\sigma}^z$ term favours classical alignment and the transverse field h on $\hat{\sigma}^x$ flips spins and so drives quantum fluctuations; we want the ground state, the lowest eigenvalue of $\hat{H}$. The variational principle says that for any normalised trial state the energy expectation is an upper bound on the true ground-state energy,

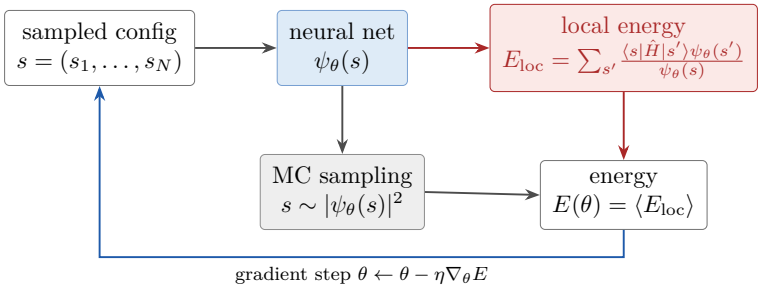
$$E[\Psi] = \frac{\langle \Psi | \hat{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle} \; \geq \; E_0, \tag{3}$$

so lowering $E[\Psi]$ over a family of trial states pushes it toward the ground state. The idea of the neural quantum state is to let that family be a neural network.

On the machine learning side, parameterise the amplitudes by a network with weights θ, written $\psi_\theta(s)$ and shown above, for example a restricted Boltzmann machine or a small feedforward net. The energy cannot be summed over all 2^N configurations, so it is rewritten as an average of a local energy over configurations sampled from the network's own probability $|\psi_\theta(s)|^2$,

$$E_{\mathrm{loc}}(s) = \sum_{s'} \frac{\langle s | \hat{H} | s' \rangle \, \psi_\theta(s')}{\psi_\theta(s)}, \qquad E(\theta) = \big\langle E_{\mathrm{loc}}(s) \big\rangle_{s \sim |\psi_\theta|^2}. \tag{4}$$

Because the Hamiltonian is local, only a handful of s' connect to each s, so E_{loc} is cheap to evaluate. The weights are then updated by gradient descent on the energy, $\theta \leftarrow \theta - \eta \nabla_\theta E(\theta)$, which is variational Monte Carlo. Iterating sampling, local-energy evaluation and the gradient step drives $E(\theta)$ down toward E_0; at small N you compare directly with exact diagonalisation, and the method then extends to sizes where exact methods fail.



In the variational Monte Carlo loop the network assigns an amplitude to each sampled spin configuration, the local energy is evaluated and averaged into an energy estimate, and its gradient updates the weights. The loop repeats until the energy converges to E_0.